



Name: _____

Reg#: _____

Date: 05-Jun-2026

Time: 120 mins

Final Exam
Computer Based
Individual proctored Assessment

BSSE23 & BSCE22

SE305T & MD445T (Spring-26)

Human Computer Interaction
Department of Computer and Software Engineering

INSTRUCTIONS

- Attempt ALL four tasks. No task is optional. Partial credit is awarded where applicable.
- Your solution must be written entirely in Python. No other programming language is permitted.
- Use Gradio (preferred) or Streamlit as the UI framework. The app must run locally via a browser at localhost.
- The chatbot must use Ollama running locally at <http://localhost:11434> via the OpenAI-compatible Python client. No external internet API is permitted.
- Verify Ollama is running and llama3 is available before starting: open a terminal and run 'ollama run llama3' to confirm. Your startup health check must also verify this automatically.
- All three modalities: **camera, voice, and text** must be functional in your final submission. A missing modality results in zero marks for that modality's sub-questions.
- Place all your code in a single folder named RegNo_Name. Run your app with python app.py before time is called. Only the running app will be evaluated.
- Hard-coding expected outputs, disabling required validations, or submitting non-functional stubs will be treated as academic dishonesty.

MARKS DISTRIBUTION

TASK	DESCRIPTION	CLO	MARKS	OBTAINED
Task 1	Multimodal Input Capture (Camera, Voice, Text)	CLO-3	15	
Task 2	Chatbot Logic, Prompt Building & Response	CLO-3	15	
Task 3	State, Error Handling & Accessibility	CLO-3	10	
Task 4	System Testing & Test Report	CLO-3	10	
TOTAL	All Tasks — Attempt All		50	

CASE STUDY — Read carefully before attempting any task**CampusConnect***AI-Powered Multimodal Student Support Chatbot · FAST University SIS***Background**

FAST University's Student Information System (SIS) receives thousands of student queries daily regarding fee payment, grade disputes, course registration errors, hostel complaints, scholarship applications, and timetable clashes. Manual help desk wait times exceed 40 minutes during peak periods. The university has commissioned a Python-based AI chatbot, **CampusConnect**, to automate first-level student support entirely.

System Requirements

CampusConnect must support three mandatory input channels simultaneously, all implemented in Python:

- **TEXT:** Students type their query in a Gradio/Streamlit Textbox (e.g., 'Why is my fee challan showing an overdue amount?'). Typed input only. A 500-character limit with live counter must be enforced in Python.
- **VOICE:** Students speak their query (e.g., 'Mujhe mera result check karna hai'). Both live microphone recording via Gradio's Audio component and pre-recorded audio file upload (.wav/.mp3) must be supported. Transcription must use Whisper (offline) and the detected text must be displayed.
- **CAMERA / IMAGE:** Students show a document, screenshot, or ID card. Both live webcam capture via Gradio's Image component (webcam mode) and image/video file upload must be supported. Captured frames are processed with OpenCV and PIL before being sent to the AI.

Chatbot Scope

The chatbot answers queries in five areas: (1) Fee & Finance, (2) Examination & Grades, (3) Course Registration, (4) Hostel & Transport, (5) Scholarship & Financial Aid. For out-of-scope queries it must politely decline and suggest the relevant office. The bot must always acknowledge which modality it primarily used to understand the query. The LLM runs locally via Ollama, no internet is needed.

Your Deliverable

A fully working Python application (app.py) that launches in the browser, implements all three input modalities, calls the Ollama local LLM API (no internet required), maintains conversation history, handles errors gracefully, and includes a written test report. No JavaScript frameworks. Python only.

STARTER CODE TEMPLATE — Provided by examiner. Copy into app.py and build on top of it.

```
#  
  
# CampusConnect — Starter Template | HCI-401 Final Exam  
# Fill every pass # TODO stub. Do NOT rename functions.  
#  
  
import gradio as gr  
import cv2  
import whisper  
import speech_recognition as sr  
import numpy as np  
from PIL import Image  
from openai import OpenAI  
import base64, io, uuid, logging, requests  
from pydub import AudioSegment  
  
logging.basicConfig(level=logging.INFO)  
logger = logging.getLogger(__name__)  
  
# — Ollama client (no internet required) —  
client = OpenAI(  
    base_url = "http://localhost:11434/v1",  
    api_key = "ollama",  
    timeout = 60.0  
)  
  
# — Whisper (loaded once — do NOT move inside a function) —  
print("Loading Whisper...")  
whisper_model = whisper.load_model("base")  
print("Whisper ready.")  
  
# — System prompt —  
SYSTEM_PROMPT = (  
    "You are CampusConnect, an AI support assistant for FAST University. "  
    "Only answer questions about: Fee & Finance, Examination & Grades, "  
    "Course Registration, Hostel & Transport, Scholarship & Financial Aid. "  
    "Politely decline anything outside scope and name the correct office. "  
    "Always state which input modality (image/voice/text) you used to understand the query."  
)  
  
# — Startup health check —  
def startup_check():  
    """TODO Task 3.B.iii — extend this with all required checks"""  
    try:  
        r = requests.get("http://localhost:11434/api/tags", timeout=3)  
        assert r.status_code == 200  
        print("\033[92m[PASS]\033[0m Ollama running")  
    except Exception as e:  
        print(f"\033[91m[FAIL]\033[0m Ollama: {e}")  
        raise RuntimeError(  
            "Ollama is not running. Open a terminal and run: ollama serve"  
)
```

```

startup_check()

# =====
# STUDENT CODE — implement every function below
# =====

def process_frame(image_np):
    """Task 1.A — receive NumPy frame, return base64 JPEG string or None"""
    pass # TODO

def transcribe_audio(audio_path):
    """Task 1.B — transcribe WAV/MP3 using Whisper, return transcript string"""
    pass # TODO

def validate_text_input(text):
    """Task 1.C — returns (is_valid: bool, error_message: str)"""
    pass # TODO

def topic_classifier(text, transcript):
    """Task 2.A — returns (in_scope: bool, topic_or_refusal_msg: str)"""
    pass # TODO

def build_message_payload(text, transcript, image_b64):
    """Task 2.A — returns OpenAI-compatible content list for one user turn"""
    pass # TODO

def trim_history(history, max_turns=10):
    """Task 2.A — keeps system prompt + last max_turns messages only"""
    pass # TODO

def handle_error(error, context=""):
    """Task 3.B — unified error handler, returns user-friendly string"""
    pass # TODO

def chat(text, audio_path, image_np, history, state):
    """Task 2.B — main callback. Returns (chatbot, history, transcript, error_msg)"""
    pass # TODO

# — Gradio UI layout (provided — do NOT remove components) —
with gr.Blocks(title='CampusConnect') as app:
    history_state = gr.State(value=[])
    with gr.Row():
        camera_input = gr.Image(
            sources=["webcam", "upload"],
            type="numpy",
            label="Camera — Live Webcam or Upload Image"
        )
        audio_input = gr.Audio(
            sources=["microphone", "upload"],
            type="filepath",
            label="Voice — Live Mic or Upload Audio"
        )

    text_input = gr.Textbox(placeholder='Type your query here...', label='Text Input', lines=3)
    char_counter = gr.Textbox(label='Character Count', interactive=False)
    transcript_box = gr.Textbox(label='Voice Transcript (auto-filled)', interactive=False)
    chatbot = gr.Chatbot(label='CampusConnect Chat')

```

```
error_label = gr.Textbox(label='Status / Error', interactive=False)
submit_btn = gr.Button('Send', variant='primary')

# live character counter — already wired for you
text_input.change(
    fn=lambda t: f'{len(t)}/500 characters',
    inputs=text_input, outputs=char_counter
)

# submit button — inputs/outputs fixed, implement chat() above
submit_btn.click(
    fn=chat,
    inputs=[text_input, audio_input, camera_input, chatbot, history_state],
    outputs=[chatbot, history_state, transcript_box, error_label]
)

# TODO Task 3 — add remaining event bindings (mode toggle, clear, etc.)
pass

if __name__ == '__main__':
    app.launch()
```

NOTE

All **pass** # **TODO** stubs are what you must implement. Do not rename any function signatures are fixed by the event bindings. You may freely add helper functions and import additional modules.

1

Task 1 — Multimodal Input Capture (CLO-3)*OpenCV · Whisper · SpeechRecognition · Gradio Components***15****MARKS****CONTEXT**

Before any chatbot logic is built, all three input pipelines must be independently functional, testable in isolation, and integrated into a single Gradio layout. Each pipeline is graded individually. A pipeline that crashes on launch earns zero for its sub-questions even if the code is partially correct.

T1.A**Camera Input Pipeline — OpenCV + Gradio**

(i) Implement a Gradio Image component configured to accept both live webcam input and image file uploads. When the student submits the form, the callback must receive the image as a NumPy array. Write the complete Gradio component definition and explain what the `type='numpy'` parameter means and why it is used here instead of `type='filepath'`.

(ii) Write a function `process_frame(image_np)` that: (a) checks if the input is `None` and returns a descriptive error string if so, (b) converts the frame from RGB (Gradio default) to BGR for OpenCV processing, (c) resizes it to a maximum of 640x480 while preserving aspect ratio using `cv2.resize()`, and (d) re-encodes it as a JPEG base64 string ready for the AI API. Show every step with the exact OpenCV and base64 calls used.

(iii) Implement a mode-toggle Radio button that switches between 'Live Webcam' and 'Upload Image/Video'. When 'Upload Image/Video' is selected, modify the Image component's sources parameter to ['upload'] only. When 'Live Webcam' is selected, restore sources to ['webcam']. In Gradio, explain how you dynamically update a component's properties after initial render using `gr.update()`.

(iv) What happens when the user submits the form without providing any camera input (`image_np` is `None`)? Write the guard clause that catches this case, prevents the API call from being made, and returns a `gr.Warning()` message visible to the user. Explain why checking for `None` before base64 encoding is critical and what error would occur without the check.

T1.B**Voice Input Pipeline — Whisper + SpeechRecognition**

(i) Implement a Gradio Audio component that accepts both live microphone recording and audio file upload (WAV and MP3). Write the `transcribe_audio(audio_path)` callback that: (a) checks if `audio_path` is `None` and returns an appropriate message, (b) loads the Whisper 'base' model (cache it at module level — explain why), (c) calls `model.transcribe()` with the file path, and (d) returns the transcript string. Why must the model be loaded once at module level rather than inside the callback function?

(ii) The Gradio Audio component returns `None` when no audio is recorded. It also returns a file path to a temporary file when audio is provided. Write the complete conditional logic that handles both cases, validates that the file exists on disk before passing it to Whisper, and explains what `FileNotFoundError` scenario could still arise even after checking for `None`.

(iii) After transcription completes, the transcript must be appended to the existing Textbox content (from Task 1.C) with a ' [voice] ' label prefix. Write the Gradio callback return statement that appends

without overwriting, given that both the transcript and the current textbox value are inputs to the function. Show the exact `gr.update()` call.

(iv) A student uploads an MP3 file but Whisper raises a `RuntimeError` because `ffmpeg` is not installed. Write the complete error handling block that: (a) catches this specific scenario, (b) attempts to detect whether the error is `ffmpeg`-related by inspecting the exception message, (c) shows a user-friendly message explaining that MP3 conversion requires `ffmpeg` and suggesting they upload a WAV file instead, and (d) logs the full traceback to the console for debugging.

T1.C**Text Input Pipeline — Gradio Textbox + Validation**

(i) Implement a Gradio Textbox with a maximum of 500 characters enforced server-side. Bind a function to the `.change()` event that: (a) returns a character count label (e.g., '312 / 500'), (b) returns `gr.update(value=text[:500])` if the text exceeds 500 characters to silently trim it, and (c) changes the counter label color to red (using Markdown) when fewer than 20 characters remain. Write all three behaviors in a single callback.

(ii) Gradio does not natively prevent copy-paste. Explain two strategies you can implement in Python/Gradio to detect and discourage pasted text: (a) comparing the change delta between two consecutive `.change()` events to flag unusually large input jumps, and (b) using a `gr.HTML()` component with a minimal JavaScript listener that disables the paste event on the specific textbox element. Show the implementation for both strategies.

(iii) Write a `validate_text_input(text)` Python function that returns `(is_valid: bool, error_message: str)`. It must reject: (a) empty or whitespace-only input, (b) input shorter than 5 characters, (c) input containing only numeric characters with no alphabetic content, and (d) input longer than 500 characters. The submit button's callback must call this function first and halt processing if `is_valid` is `False`, showing the `error_message` to the user.

2

Task 2 — Chatbot Logic & AI Integration (CLO-3)*Prompt Engineering · Ollama Local LLM · OpenAI-compatible Client · Conversation History***15
MARKS****CONTEXT**

The chatbot must intelligently fuse all three inputs into a structured API call, maintain multi-turn context, enforce topic scope, and stream responses back to the user. This task is the core of the system and carries the highest marks.

T2.A**Prompt Construction & Multimodal Fusion**

(i) Write a `build_message_payload(text, transcript, image_b64)` Python function that constructs the messages list for one user turn in the OpenAI-compatible format used by Ollama. The function must: (a) always include the typed text as a text content block, (b) append the transcript as a separate labeled text block only if transcript is not empty, (c) prepend the base64 image block (type: `image_url`) only if `image_b64` is not None, and (d) return the content list in the order: image → transcript block → text block. Show the exact dict structure for each block type and the final messages list passed to `client.chat.completions.create()`.

(ii) Write the system prompt string for CampusConnect. It must: (a) define the bot's persona and name, (b) explicitly list the five topic areas it may answer, (c) instruct it to refuse off-topic queries politely and name the correct office, (d) instruct it to always begin its response by acknowledging which input modality was most informative (e.g., 'Based on the image you shared...'), and (e) instruct it to respond in the same language the student used (Urdu or English). Write the full system prompt as a Python string.

(iii) Implement a local `topic_classifier(text, transcript)` function that runs before any API call. It must use a keyword dictionary mapping each of the five topic areas to at least five keywords each. If neither the text nor the transcript matches any topic, the function returns (False, 'off-topic message string'). If a match is found, it returns (True, `topic_name`). Show the full keyword dict and the matching logic. Explain why running this locally saves API tokens.

(iv) The API context window has a token limit. Implement a `trim_history(history, system_prompt, max_turns=10)` function that: (a) always keeps the system prompt at position 0, (b) keeps only the last `max_turns` message dicts, and (c) logs a warning to the console when trimming occurs. Show where in the main chat function you call `trim_history` before passing history to the API.

T2.B**API Call, Streaming & Response Display**

(i) Write the complete `chat(text, audio_path, image_np, history, session_id)` Gradio callback function. It must: (a) validate all inputs using Task 1 validators, (b) process image via `process_frame()`, (c) transcribe audio via `transcribe_audio()`, (d) classify topic, (e) build the full messages list using `build_message_payload()`, (f) call the Ollama API via the OpenAI-compatible client at `http://localhost:11434/v1`, (g) append both user and assistant turns to history, and (h) return the updated history to the Chatbot component and updated `gr.State`. Show the complete function signature and body including the client initialization.

(ii) Implement streaming for the Ollama response. Pass `stream=True` to `client.chat.completions.create()`. Write a Python generator function that iterates over the streaming response chunks, extracts `chunk.choices[0].delta.content`, and yields the progressively accumulated response string. Explain how Gradio's generator-based streaming works: what must the function use instead of `return`, and how does Gradio know to update the Chatbot component on each `yield`?

(iii) Implement a 'Copy Response' button next to each AI message. Since Gradio does not natively support per-bubble copy buttons, implement this by adding a `gr.HTML()` component that renders a small copy icon using JavaScript's `navigator.clipboard.writeText()`. The text to copy must be passed from Python to the HTML component via `gr.update(value=html_string)` on each new response. Show the complete implementation.

(iv) Implement a 'Clear Conversation' button that: (a) resets the history `gr.State` to an empty list, (b) clears the Chatbot display, (c) clears the Textbox, (d) resets the Image component to `None`, (e) resets the Audio component to `None`, and (f) generates a new `session_id` in the session State. Write the `clear_all()` function with the correct tuple of return values matching the outputs list of the button's click event.

(v) After the AI responds, parse the response text to extract which modality it claims to have used (look for phrases like 'Based on the image', 'From your spoken query', 'From your typed message'). Write a `parse_modality_used(response_text)` function that returns one of: 'image', 'voice', 'text', or 'combined'. Use this result to update a `gr.Textbox` label in the UI showing 'Primary modality understood: image' so the student always knows how the bot interpreted their input.

3

Task 3 — State Management, Error Handling & UX (CLO-3)*Robustness · Graceful Degradation · Accessibility***10
MARKS****CONTEXT**

A multimodal Python app has many failure modes: missing ffmpeg, Whisper model not downloaded, camera index wrong, Ollama server not running, model not pulled, slow inference timeout, None inputs from Gradio. This task tests how comprehensively you handle all of these without crashing the app.

T3.A**Application State with gr.State()**

- (i) Define the complete initial application state as a Python dictionary stored in a `gr.State()` component. It must include: `session_id` (uuid), `conversation_history` (list), `is_processing` (bool), `last_error` (str or None), `camera_mode` ('webcam' or 'upload'), `voice_mode` ('mic' or 'upload'), and `char_count` (int). Explain why `is_processing` must be part of State rather than a module-level global when multiple students use the app simultaneously.
- (ii) Write an `update_state(current_state, **kwargs)` helper function that merges keyword argument updates into the state dict and returns the new dict. Demonstrate its use: show how a single line in the submit callback can set `is_processing=True` and update `last_error=None` simultaneously. Explain why always returning a new dict (rather than mutating in place) is safer in a concurrent Gradio environment.
- (iii) Implement a submit guard: at the very start of the `chat()` callback, check `state['is_processing']`. If True, immediately return all current outputs unchanged plus a `gr.Warning('Please wait for the current response to complete.')`. Set `is_processing=True` in the state before the API call and False in a finally block. Write the complete try/finally structure that guarantees `is_processing` is reset even if an exception occurs mid-call.
- (iv) When the user switches `camera_mode` from 'webcam' to 'upload' via a Radio button, the Image component must update its sources. Write the `mode_switch(selection, state)` callback that: (a) updates `camera_mode` in state, (b) returns `gr.update(sources=['webcam'])` or `gr.update(sources=['upload'])` based on selection, and (c) also resets the current image value to None so the old capture is not accidentally submitted with the new mode. Show the full callback and its Gradio event binding.

T3.B**Error Handling & Graceful Degradation**

- (i) Write a unified `handle_error(error: Exception, context: str)` Python function that: (a) logs the full traceback using `logging.exception()`, (b) maps known exception types to user-friendly messages: `requests.exceptions.ConnectionError` → 'Ollama server is not running. Please start Ollama and try again.', `openai.APITimeoutError` → 'The model is taking too long to respond. Try a lighter model like mistral.', `openai.APIConnectionError` → 'Cannot reach the local AI server at localhost:11434.', `sr.UnknownValueError` → 'Could not understand audio, please try again.', `FileNotFoundError` → 'Audio or image file not found or corrupted.', `cv2.error` → 'Camera error: check your device connection.', and (c) returns a safe default string for any other Exception. Show the complete function with all branches.
- (ii) Implement an exponential backoff retry decorator `@retry_with_backoff(max_retries=3, base_delay=2)` for the Ollama API call. The decorator must: (a) retry only on

`openai.APIConnectionError` and `openai.APITimeoutError` (both mean the server is busy or slow, not a permanent error), (b) wait `base_delay ** attempt` seconds between retries, (c) update a `gr.Textbox` label with 'Retrying in Xs...' countdown before each wait, and (d) raise the final exception after `max_retries` are exhausted. Write the complete decorator.

(iii) On application startup (before `gr.Blocks()` launches), perform a system health check: (a) verify Ollama is running by sending GET `http://localhost:11434/api/tags` and checking for a 200 response, (b) verify the 'llama3' model is available by checking that 'llama3' appears in the tags response JSON, (c) verify Whisper can be imported and the 'base' model file exists in cache, (d) verify OpenCV can open camera index 0. Print a color-coded PASS/FAIL report to the console for each check. If Ollama or llama3 is not found, print a clear instruction ('Run: ollama pull llama3') and raise a `RuntimeError` before the app starts.

T3.C**UX Polish & Accessibility**

(i) Implement a status indicator panel using a `gr.Dataframe` or three `gr.Textbox` components (one per modality) that always shows the current status of camera (Active / Off / Error), voice (Transcribing / Ready / Error), and text (X/500 chars). This panel must update after every user action. Write the `update_status_panel(state)` function that reads from the state dict and returns the three status strings.

(ii) Gradio automatically generates some accessibility attributes, but you must ensure: (a) every `gr.Button` has a descriptive `label=` parameter (not just an icon), (b) every `gr.Image` and `gr.Audio` has a `label=` parameter that clearly describes its purpose, and (c) the `gr.Chatbot` component has `show_label=True` with a meaningful label. Rewrite the component definitions from your Task 1 implementation to include all required accessibility labels.

(iii) Implement a 'dark mode / light mode' toggle using Gradio's built-in theme system. Add a `gr.Button('Toggle Theme')` that switches between `gr.themes.Default()` and `gr.themes.Monochrome()`. Since Gradio themes are set at app launch, explain the alternative approach: use a `gr.HTML()` component to inject a CSS class toggle on the root element via JavaScript, and provide the minimal CSS needed to invert background and text colors. Show both the Python Gradio approach and the JavaScript fallback.

4

Task 4 — Final System Testing & Documentation (CLO-3)
Integration Tests · Edge Cases · Written Test Report
10
MARKS
CONTEXT

You must design, execute, and honestly document test cases for your own application. Fabricated results earn zero. Failures with correct root cause analysis and proposed fixes earn full marks for that row. The test report is a real engineering document.

T4.A**Required Test Cases — Execute and Document**

(i) Design and execute all EIGHT required test cases below. For each, document in a table: Test ID, Precondition, Steps Executed, Expected Result, Actual Result, Pass / Fail. TC-01: Submit with only typed text (no audio, no image). TC-02: Submit with only a webcam-captured image (no text, no audio). TC-03: Submit with all three modalities simultaneously. TC-04: Submit with an empty Textbox while audio and image are provided. TC-05: Upload an MP3 audio file and verify transcription or graceful error. TC-06: Submit while `is_processing` is True (rapid double-click simulation). TC-07: Type exactly 501 characters and verify the 500-char limit is enforced. TC-08: Switch from webcam to image upload mid-session and verify the previous capture is cleared.

(ii) Design and execute FOUR additional edge-case test cases of your own choosing. Each must target a different failure mode not covered by TC-01 to TC-08 (examples: Ollama server stopped mid-session, Whisper model file deleted after app start, camera disconnected mid-session, history exceeding 10 turns triggering the sliding window trim). Document in the same table format.

(iii) For every test case that received a FAIL result: write (a) the Root Cause — one sentence identifying the exact function name, line number or logic, and reason for failure, and (b) the Proposed Fix — one sentence or a code snippet of max 5 lines showing the correction. Do not leave any FAIL row without analysis. A blank analysis for a FAIL earns zero for that row.

T4.B**Written Test Report**

(i) Write a structured one-page test summary report containing: (a) Total test cases designed, Pass count, Fail count, Partial count. (b) The single most critical unresolved defect: its Test ID, description, severity (Critical / Major / Minor), and the direct user impact in one sentence. (c) One concrete recommendation for the team before production deployment. (d) Your overall system confidence rating (1–5) with a two-sentence justification.

(ii) Identify the single most significant HCI design weakness in your implementation — not a code bug, but a design decision that reduces usability for the target user (FAST University students). State: (a) HCI principle is violated, (b) the exact element in your UI that violates it, (c) a specific, implementable design change that would fix it, and (d) how you would verify the fix improved usability (one-sentence test method).